3S – Secure Streaming Solution (Part I):

Secure Channel, Handshake and Flexible Cryptographic Configurations for Real-Time MPEG-4 Streaming

Student 79449 and Student 72966

Computer Systems Security, 2026/2027, 1st Semester – Work Assignment #1

Pair Programming

1 Introduction

The provided baseline consists of an `OriginSocketServer` (Stream Server) that serves MP4 files in 1 KiB blocks over raw TCP sockets and a `ProxySocketServer` (Proxy) that relays them to a browser, honoring HTTP `Range` requests. The baseline offers no secrecy, integrity, peer authentication or replay protection.

The objective of Work Assignment 1 is to build a secure channel between Proxy and Stream Server with confidentiality, message integrity and authenticity, replay protection and forgery protection, negotiated by a handshake and fully configurable without recompilation. Our work so far covers the part that carries the security argument: the cryptographic configuration layer, the handshake and the protected message channel, together with an adversarial test bench.

2 System Model and Architectural Design

The browser speaks plain HTTP to the local Secure Proxy (which runs on the viewer's trusted host). The Proxy opens a TCP connection to the Secure Stream Server and runs the SSP over it. The Authentication Server belongs to Work Assignment 2 and is not part of this deliverable.

[Insert Figure 1 here: Web browser → (HTTP) → Secure Proxy ↔ (SSP/TCP, secure channel) ↔ Secure Stream Server → MP4 files; Auth. Server (WA2) shown dashed. The SSP library (`common/`) is shared by Proxy and Server.]

The shared library contains `CryptoConfig` (parsing and validation), `CryptoSuite` (encryption/decryption of one message), `Handshake` (types 1–3), `SecureChannel` (framing, types 4/5/255, sequence numbers, AAD) and typed exceptions mapped to alert codes.

Table 1: Implementation status at the time of writing.

Component	Content	Status
`CryptoConfig`	Parsing (hardened against XXE), hex/Base64, validation	Done, tested
`CryptoSuite`	AEAD and Encrypt-then-MAC, random nonce per message, AAD	Done, tested
`Handshake`	Types 1–3, SRV challenge, config confirmation	Done, tested
`SecureChannel`	Header framing, types 4/5/255, SEQ checks, byte	Done, tested

Component	Content	Status
	counters	
Secure Proxy / Server	HTTP↔SSP integration, metrics, range handling	**Pending** (baseline still plaintext)
DH key agreement	Authenticated ephemeral key exchange, forward secrecy	**Pending** (pre-shared keys)
Extensions	Concurrency tests, Docker	Pending

3 Threat Model and Security Countermeasures

A network attacker ("Mallory") controls the path between Proxy and Server: she can eavesdrop, modify, inject, replay, reorder, duplicate and drop messages, and can open her own connections to the Server. Both endpoints are uncompromised; the primitives (AES, ChaCha20, Poly1305, GCM, HMAC-SHA2, SHA-256) are secure when correctly used; the configured keys are secret and random. Endpoint compromise, breaking primitives and traffic analysis (message sizes and timing) are out of scope; implementation errors, bad configurations and protocol misuse are in scope.

Table 2 maps threats to mechanisms. We report honestly the two requirements that the present implementation does not meet.

Table 2: Threat-to-mechanism mapping.

Threat	Mechanism	Status
Eavesdropping on media	AEAD or cipher+HMAC; fresh random nonce per message	Mitigated
Tampering with a block	GCM/Poly1305 tag or HMAC over aad‖nonce‖ct; any bit flip rejected (INTEGRITY_FAILURE)	Mitigated
Forgery	Requires the secret key(s); verification before use (HMAC checked before decryption; constant-time compare)	Mitigated (given key secrecy)
Replay / duplication	Receiver accepts only the next expected SEQ; SEQX random (62 bit); fresh 256-bit SRV per handshake; R = SHA-256(SRV)	Mitigated
Reordering, dropping	Strict SEQ+1 rule; SEQ also in AAD, so rewriting the clear SEQ fails the tag	Mitigated
Truncation / block-END confusion	Authenticated end block (type 5); EOF before it is an error; version, release, type in AAD	Mitigated
Altered / downgraded proposal	Server accepts only a configuration identical to its own; confirmation CCC returned inside encrypted	Mitigated

Threat	Mechanism	Status
	type 2 and compared by the Proxy	
Weak configuration (ECB, no MAC, same key for cipher and MAC, wrong key size)	Rejected at load time by `validate()`	Mitigated
Resource exhaustion by forged size	Payload size capped at 1 MiB before allocation	Mitigated
Man-in-the-middle on key agreement	Peers authenticated only implicitly, by possession of the pre-shared key. No DH exchange yet.	**Partial**
Session-key compromise / forward secrecy	Keys are static: a leaked key exposes all recorded and future sessions. Not provided.	**Not met**
Forged alerts (type 255) / connection reset	Alerts are unauthenticated; an on-path attacker can already cut the connection	Out of scope
User authentication	Authentication Server	WA2

4 Specification of the Secure Streaming Protocol

Every message is Header‖Payload. The header is in clear: version (2 B), release (2 B, currently 1:1), type (1 B) and payload size (4 B), i.e. 9 bytes. Sequence numbers are 8 bytes. E_K(·) denotes the configured protection of one message (Sect. 5); aad_t = ver‖rel‖t for type t.

Handshake. The Proxy sends SEQX and the proposal PCC taken from `proxycrypto.config`. The Server compares it with `sservercrypto.config`; if incompatible it answers alert CONFIG_REJECTED. Otherwise it returns, encrypted, SEQY=SEQX+1, a 256-bit nonce SRV from `SecureRandom` and the confirmation CCC. The Proxy checks the sequence number and that CCC equals what it proposed (type 1 travels in clear, so this encrypted echo is what exposes a modified proposal). Message 3 carries SEQZ=SEQY+1, R=SHA-256(SRV) and the movie name; the Server verifies R in constant time and only then streams.

Streaming. Blocks are sent as SEQB‖E_K(block) with SEQB starting at SEQZ+1, with the clear SEQB also bound in the AAD (aad_t‖SEQB). The receiver checks the SEQ first (cheap rejection of replays) and then authenticates and decrypts. The last block is type 5 with SEQF and an encrypted "END‖random" body; a connection that ends before a valid type 5 is an error. SEQF is authenticated through the AAD instead of being repeated inside the plaintext body; the guarantee is equivalent.

Alerts. Type 255 carries a code (1 byte) and a reason: 1 unexpected message, 2 bad version, 3 malformed, 4 integrity failure, 5 bad sequence, 6 configuration rejected, 7 bad challenge, 8 media not found. Decryption failures always return the same message, to avoid giving an oracle to the attacker.

5 Cryptographic Configuration Files

Both `proxycrypto.config` and `sservercrypto.config` follow the XML layout of the statement (SymCipherSuite, MACsuite, ConfiguredKeys); NULL marks an unused field. Nothing cryptographic is hard-coded: the JCE transformation, IV size, MAC name and keys are all derived from the file. Tag names are matched case-insensitively and keys are accepted in hex or Base64 with their declared size checked.

Message protection. AEAD modes (GCM, Poly1305, CCM) [1, 2, 3] output nonce‖ct‖tag with the AAD passed to the cipher. Other modes use Encrypt-then-MAC with an independent key: nonce‖ct‖HMAC_K2(aad‖nonce‖ct) [4]. A fresh random nonce is drawn for every message. Five configurations are provided and tested (Table 3); the IV size is taken from the file (96 bit for GCM/ChaCha20, 128 bit for CBC/CTR).

Validation at load time. The loader refuses: ECB; any non-AEAD mode without a MAC; a MAC without a key; a MAC key equal to the encryption key; key sizes other than 128/192/256 bits; a key whose length disagrees with its declared size; a mode with no IV size. Five invalid example files (`examples/INVALIDO-*.config`) exercise these rules. One case (ChaCha20 with a 128-bit key) is currently detected at first use rather than at load, a small improvement still to make.

6 Secure Proxy and Stream Server: Design and Implementation Options

Status: pending integration. `ProxySocketServer` and the origin server are still the baseline plaintext code.

Proxy. For each browser request the Proxy opens a TCP connection to the Server, runs `Handshake.proxy()` with the file name from `/video/<name>`, then loops on `receive()`/`openSeqClear()` writing each decrypted block to the browser until type 5. Any `ProtocolException` sends the corresponding alert and aborts the stream, so the browser never receives unauthenticated bytes. Browsers issue several Range requests, so each upstream request is a separate SSP session with its own handshake, SRV and SEQX, which keeps replay protection per session. Since type 3 carries only the movie name, the Range and the resulting Content-Length/Content-Range must travel inside protected fields (the MEDIA string in type 3 and the first encrypted block of the reply), which keeps them authenticated.

Server. It accepts a connection, runs `Handshake.server()` against `sservercrypto.config`, resolves the requested name against the media directory (rejecting paths outside it, with alert 8 if absent), and sends the file in 1 KiB blocks with `sendSeqClear(TYPE_4, ...)` starting at `firstBlockSeq`, closing with a type-5 block. Since the baseline already uses a cached thread pool and the `CryptoSuite` and counters are per connection, concurrent sessions should work independently; this still has to be tested.

Instrumentation. `SecureChannel` already counts bytes on the wire. The Proxy will add counters for received, decrypted-and-forwarded blocks and plaintext bytes forwarded, and compute throughput as plaintext bytes divided by the time from sending type 1 to receiving type 5 (`System.nanoTime`), reported in MB/s together with the original file size.

7 Validation

7.1 Testbench

The tests are Java programs in `common/` and `tools/`, with raw outputs saved in `results/`. `CryptoSuiteTest`: round-trip, nonce uniqueness, exhaustive single-bit/AAD/truncation tampering and per-1 KiB overhead. `ChannelTest` (per configuration): a recorded session (type 2, three type-4 blocks, type 5) is manipulated by Mallory and fed to a receiver. `HandshakeTest`: real TCP sockets with a relay in the middle that applies the attack. All use the five example configurations and the default AES-GCM-256.

7.2 Results

Cryptographic suite. For all five configurations the round trip succeeds, each message gets a new nonce and every modification is rejected. Table 3 lists the measured expansion per 1 KiB message and the per-block wire cost (adding the 9 B header and 8 B SEQ).

Table 3: Tested configurations: measured expansion of a 1 KiB message and derived per-block wire overhead.

Configuration	Nonce/IV	Suite expansion	Wire overhead per block
AES/GCM/NoPadding (96-bit IV)	12 B	+28 B (2.7%)	45 B (4.4%)
ChaCha20-Poly1305	12 B	+28 B (2.7%)	45 B (4.4%)
ChaCha20 + HMAC-SHA256	12 B	+44 B (4.3%)	61 B (6.0%)
AES/CTR + HMAC-SHA256	16 B	+48 B (4.7%)	65 B (6.3%)
AES/CBC/PKCS5 + HMAC-SHA-512	16 B	+96 B (9.4%)	113 B (11.0%)

Channel attacks. In each of the five configurations all 12 scenarios behave as required (60/60). The normal session is accepted; a replayed, swapped, dropped or injected-from-another-session block is rejected with BAD_SEQUENCE; a dropped block whose clear SEQs were "repaired", a block turned into an end-of-stream (type 4→5) and a single flipped ciphertext bit are rejected with INTEGRITY_FAILURE; a changed version gives BAD_VERSION; a forged 2 GB size gives MALFORMED (no allocation); and a missing end block or cut connection is detected as premature end of stream. The "repaired SEQ" case shows the value of binding SEQ in the AAD.

Handshake (AES-GCM, real sockets). All 7 scenarios pass: normal handshake (both sides obtain the same first-block SEQ and movie name); a Proxy proposing ChaCha20/Poly1305 against an AES-GCM Server (CONFIG_REJECTED); same configuration but different key (INTEGRITY_FAILURE); Mallory altering the PCC (CONFIG_REJECTED) or SEQX (BAD_SEQUENCE); replay of old messages 1 and 3 (BAD_CHALLENGE, since SRV is fresh); and message 3 sent first (UNEXPECTED_MESSAGE).

8 Conclusions and Future Work

We specified and implemented the security-critical core of 3S Part I: a configurable cipher-suite layer, an SSP handshake that detects modified proposals, wrong keys and replays, and a block channel that

detects tampering, forgery, replay, reordering, dropping and truncation. All 60 + 7 + 10 recorded tests pass. The result is not yet a full solution: the Proxy and Server executables must be integrated and measured, and the following security gap must be closed.